UNIT 2 · TOPIC 2.4

Detecting Physical Attacks

Assignment, Answer Key

Teacher answer key, 30 points total.

Practice Questions

2.4.a · Access logs and monitoring

1. An access log shows badge #1042 granted entry to the server room at 02:17 on a Saturday. What is the PRIMARY concern? (1 pt)

- ○ A. The badge reader malfunctioned overnight
- ○ B. Anti-passback was not enabled on that reader
- ○ C. The SIEM platform failed to generate an alert
- **● D. After-hours access to a restricted area by a valid credential**

Why: After-hours valid access to a restricted door is an alert-worthy anomaly regardless of credential validity, it may indicate a stolen or cloned badge.

2. Which of the following log patterns are direct indicators of a potential physical security breach? Select ALL that apply. (2 pts)

- **■ A. Seven ACCESS DENIED events for badge #0099 within 45 seconds**
- □ B. Badge #0210 granted at lobby at 08:32, then at server room at 08:34
- **■ C. Badge #0210 granted at lobby at 08:32, then at data center 40 miles away at 08:45**
- □ D. Badge #0055 granted at lobby at 09:05 on a Tuesday

Why: Seven denials in 45 seconds is a repeated-denial burst (rule 0). The same badge appearing 40 miles away within 13 minutes is a location inconsistency (rule 2). The other events are normal.

3. What does the anti-passback feature in an access control system detect? (1 pt)

- ○ A. Intrusions through windows and emergency exits
- ○ B. After-hours access by cleaning staff
- **● C. A credential being used to enter without a prior exit record, suggesting badge sharing**
- ○ D. Repeated failed attempts on a single reader

Why: Anti-passback tracks entry/exit state per credential. A badge that enters twice without an exit in between triggers an alert, indicating the badge may have been passed to a second person.

4. Name the type of platform that aggregates access logs from multiple readers and applies detection rules in real time. (1 pt)

Acceptable answers: siem; security information and event management; siem platform

Why: A SIEM (Security Information and Event Management) platform centralizes log data and evaluates rules to generate alerts.

5. Why are physical access logs described as a detective control rather than a preventive control? (1 pt)

- ○ A. They are only reviewed by auditors once a year
- ○ B. They block a badge once it has been used too many times
- **● C. They record events after they occur and help discover breaches rather than block them**

○ D. They can be deleted by administrators, so they cannot be trusted

Why: A log writes down what already happened, so its job is to detect and investigate (detective), not to stop entry (preventive, like the lock or badge reader itself). Logs can feed real-time alerts, but they still react to events. Protecting logs from tampering is a separate integrity concern.

2.4.b · Alarm systems

6. A homeowner arms their system in ARMED_AWAY mode and the front-door contact sensor trips 10 seconds later while they are still walking out. What should happen? (1 pt)

○ A. The panel immediately dispatches police

○ B. The siren sounds locally but the sensor is permanently disabled

● C. The trip is treated as expected behavior because it falls within the exit delay

○ D. The system automatically switches to ARMED_STAY

Why: The exit delay gives the occupant a grace period to leave after arming; a trip inside that window is expected, not an intrusion.

7. Why is an armed alarm system classified as a detective control rather than a preventive control? (1 pt)

○ A. It only works when someone is home to hear the siren

● B. It notices and reports an intrusion in progress but cannot physically stop entry

○ C. It deters burglars, which makes it a deterrent control instead

○ D. It is only triggered after damage has been done

Why: An alarm senses an intrusion and raises the alarm; the door is still physically openable. That makes it detective, and its value depends on the response it triggers (monitoring center, police). It can also deter, but its main function is detection, and it triggers during the intrusion, not only after damage.

8. Which of the following are true about panic/duress buttons? Select ALL that apply. (2 pts)

□ A. They are always triggered by a burglary in progress

■ B. They can be used for personal-safety emergencies unrelated to a break-in

■ C. They bypass normal arm-state checks

■ D. They typically escalate straight to dispatch without cross-zone verification

Why: Panic buttons are general duress signals (medical emergency, threat, robbery, etc.), work regardless of arm state, and escalate directly to dispatch since a human deliberately triggered them.

9. What is the purpose of cross-zone verification in an alarm system? (1 pt)

○ A. To extend the exit delay automatically

○ B. To disable sensors that trip too often

● C. To require two independent sensors/zones to trip within a short window before treating the event as high-confidence

○ D. To identify which employee armed the system

Why: Cross-zone verification reduces false dispatches by requiring corroboration from a second independent sensor or zone before an event is treated as a likely real intrusion.

10. Name the alarm system component that is a staffed facility receiving panel signals and deciding whether to contact police. (1 pt)

Acceptable answers: monitoring center; central station; monitoring station; central monitoring station

Why: The monitoring / central station receives the signal, attempts verification, and dispatches when warranted.

11. A door contact sensor trips 900 seconds after the system was armed, with no other zone corroborating the event within the verification window. How should this be classified? (1 pt)

- **● A. LOCAL_ONLY**
- ○ B. FALSE_ALARM
- ○ C. SUPPRESSED
- ○ D. DISPATCH

Why: Well past the exit delay, so it is a real trip, but with no cross-zone corroboration it stays LOCAL_ONLY rather than auto-escalating to dispatch.

2.4.c · Motion detectors and sensor-based detection

12. What does a passive infrared (PIR) sensor actually detect? (1 pt)

- ○ A. A reflected microwave signal bounced off a moving object
- ○ B. The exact body temperature of anyone in the room
- **● C. A change in infrared (heat) radiation across segments of its field of view**
- ○ D. Sound waves generated by footsteps

Why: PIR sensors are passive; they emit no energy, and detect a change (delta) in infrared radiation as a heat source moves across detection segments, not an absolute temperature reading.

13. A microwave motion sensor detects movement using which underlying physics principle? (1 pt)

- ○ A. Infrared radiation change
- **● B. Doppler shift in a reflected signal**
- ○ C. Barometric pressure change
- ○ D. Capacitive touch sensing

Why: Microwave (and ultrasonic) sensors emit a signal and measure the frequency shift, Doppler shift, in the reflection caused by a moving object.

14. Why do dual-technology sensors (PIR + microwave) reduce false positives compared to a single-technology sensor? (1 pt)

- **● A. They require both technologies to indicate motion at roughly the same time (AND-logic), and the two technologies' false-trigger causes rarely overlap**
- ○ B. They trigger when EITHER technology sees motion, which doubles the coverage
- ○ C. Microwave signals cancel out the heat noise that confuses PIR sensors
- ○ D. They only arm at night, when false triggers are rare

Why: Heat from vents or sunlight fools PIR; moving curtains or fans fool microwave. Requiring BOTH to agree (AND-logic) means one technology's false trigger is not enough. OR-logic would do the opposite: catch more motion but also MORE false alarms.

15. Which of the following are known causes of motion-sensor false positives? Select ALL that apply. (2 pts)

- **■ A. Sunlight moving across a window**
- □ B. A person walking directly toward the sensor
- **■ C. HVAC vent airflow**
- **■ D. A pet moving through the room**

Why: Sunlight, HVAC airflow, and pets are all classic false-positive causes for motion sensors. A person walking through the zone is exactly the true detection the sensor exists to catch.

16. A dual-technology sensor reports: PIR element cause = 'hvac_airflow', microwave element cause = 'person'. What should the system conclude? (1 pt)

○ A. TRUE DETECTION, because the microwave element saw a person

○ B. TRUE DETECTION, because at least one element always overrides the other

● C. FALSE POSITIVE, because AND-logic requires both elements to independently indicate a person, and the PIR element's cause is a known false-positive

○ D. The event should be discarded and never logged

Why: Dual-tech AND-logic requires BOTH elements to agree the cause is a genuine person. Since the PIR element's cause is HVAC airflow (a known false-positive cause), the event is a false positive even though the microwave element reported a person.

17. What is the main risk of setting a motion sensor's sensitivity too low? (1 pt)

○ A. It will trigger constantly on pets and insects

● B. It may miss a genuine slow or partial intrusion, such as someone reaching through a cracked window

○ C. It will raise the false-positive rate from HVAC airflow

○ D. It will make cross-zone verification unnecessary

Why: Lower sensitivity reduces false alarms but creates false negatives: small or slow movements stop registering, and careful intruders move slowly on purpose. Constant triggering on pets and airflow is the problem of sensitivity set too HIGH. Tuning sensitivity is always a trade-off between the two errors.

18. Name the term for the limited field of view a motion sensor can actually observe, outside of which (or behind an obstruction) it cannot detect motion. (1 pt)

Acceptable answers: detection cone; coverage pattern; field of view; coverage cone

Why: A sensor's detection cone (or coverage pattern) is analogous to a camera's field of view, anything outside it is a blind spot.

Coding Assignments (Python 3)

2.4.a · Access logs and monitoring

3 pts · auto-graded on the site

Coding problem 2.4.a: Access log anomaly detector

Skill: Detect Attacks · **Maps to:** 2.4.A · **Language:** Python 3 (pytest)

Problem

You are building the detection engine for a physical access control monitoring system at Crestfield Analytics. Given a list of timestamped access log entries, identify every badge ID that triggers at least one of three alert rules:

- **Rule 0, Repeated denial burst:** 5 or more ACCESS DENIED events for the same badge within any 60-second sliding window.
- **Rule 1, After-hours valid access to a restricted door:** any GRANTED entry at a door whose name starts with `"restricted-"` between 22:00 and 05:59 (inclusive, using the entry's local time).
- **Rule 2, Location inconsistency:** the same badge granted at two different doors within a gap of 10 minutes (600 seconds) where those two doors are listed as geographically separated in a provided `far_pairs` set of frozensets.

Function contract

Implement `solve(entries, far_pairs)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `entries`: `list[dict]`, each dict has: - `"ts"`: `int`, Unix timestamp (seconds) - `"badge"`: `str`, badge identifier e.g. `"B-0101"` - `"door"`: `str`, door identifier e.g. `"restricted-server-room-A"` - `"outcome"`: `str`, `"GRANTED"` or `"DENIED"`
- `far_pairs`: `set[frozenset]`, each element is a frozenset of two door names that are geographically far apart.

Output

A **sorted list of unique badge ID strings** that triggered one or more rules. Empty list if no alerts.

Examples

```
entries = [
    {"ts": 1710230400, "badge": "B-0101", "door": "lobby", "outcome": "DENIED"},
    {"ts": 1710230410, "badge": "B-0101", "door": "lobby", "outcome": "DENIED"},
    {"ts": 1710230420, "badge": "B-0101", "door": "lobby", "outcome": "DENIED"},
    {"ts": 1710230430, "badge": "B-0101", "door": "lobby", "outcome": "DENIED"},
    {"ts": 1710230440, "badge": "B-0101", "door": "lobby", "outcome": "DENIED"},
]
far_pairs = set()
solve(entries, far_pairs)  ->  ["B-0101"]

entries2 = [
    {"ts": 1710288001, "badge": "B-0202", "door": "restricted-vault", "outcome": "GRANTED"},
]
# ts 1710288001 = 2024-03-13 02:00:01 UTC  (after-hours)
far_pairs2 = set()
solve(entries2, far_pairs2)  ->  ["B-0202"]
```

Constraints

- Deterministic, offline, standard library only.
- Treat all timestamps as UTC when checking hour for after-hours (hour in {22,23,0,1,2,3,4,5}).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Set, FrozenSet, Dict
import datetime


def solve(entries: List[Dict], far_pairs: Set[FrozenSet]) -> List[str]:
    flagged = set()

    by_badge: Dict[str, List[Dict]] = {}
    for e in entries:
        by_badge.setdefault(e["badge"], []).append(e)

    after_hours = {22, 23, 0, 1, 2, 3, 4, 5}

    for badge, evts in by_badge.items():
        evts_sorted = sorted(evts, key=lambda x: x["ts"])

        denials = [e["ts"] for e in evts_sorted if e["outcome"] == "DENIED"]
        for i in range(len(denials)):
```

```
            window_end = denials[i] + 60
            count = sum(1 for ts in denials[i:] if ts <= window_end)
            if count >= 5:
                flagged.add(badge)
                break

        for e in evts_sorted:
            if e["outcome"] == "GRANTED" and e["door"].startswith("restricted-"):
                hour = datetime.datetime.utcfromtimestamp(e["ts"]).hour
                if hour in after_hours:
                    flagged.add(badge)
                    break

        granted = [e for e in evts_sorted if e["outcome"] == "GRANTED"]
        for i in range(len(granted) - 1):
            a, b = granted[i], granted[i + 1]
            if a["door"] != b["door"] and (b["ts"] - a["ts"]) <= 600:
                pair = frozenset({a["door"], b["door"]})
                if pair in far_pairs:
                    flagged.add(badge)
                    break

    return sorted(flagged)
```

2.4.b · Alarm systems

3 pts · auto-graded on the site

Coding problem 2.4.b: Alarm event classifier

Skill: Detect Attacks · **Maps to:** 2.4.B · **Language:** Python 3 (pytest)

Problem

You are building the triage logic for an alarm control panel at Crestfield Analytics. Given a timestamp-ordered stream of sensor events (one per zone trip), classify each event using the arm-state, exit-delay, and cross-zone verification rules taught in the lesson:

- **SUPPRESSED**: the system was `"DISARMED"` when the event occurred (does not apply to panic buttons, see below).
- **FALSE_ALARM**: the event's sensor is `"contact"` or `"glass_break"`, the system was armed, and the event happened less than `exit_delay_sec` seconds after the zone was armed (still inside the exit-delay grace period).
- **LOCAL_ONLY**: a real trip (armed, past the exit delay) that has no corroborating trip in a *different* zone within `verify_window_sec` seconds. Only the local siren sounds.
- **DISPATCH**: either (a) the sensor is `"panic"` (a panic/duress button always dispatches, regardless of arm state or timing), or (b) a real trip that has a corroborating real trip in a different zone within `verify_window_sec` seconds (cross-zone verification).

Function contract

Implement `solve(events, exit_delay_sec=45, verify_window_sec=120)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `events`: `list[dict]`, each with: - `"ts"`: `int`, Unix timestamp (seconds) the event occurred - `"zone"`: `str`, zone/door identifier, e.g. `"front-door"` - `"sensor"`: `str`, one of `"contact"`, `"glass_break"`, `"panic"` - `"arm_state"`: `str`, one of `"DISARMED"`, `"ARMED_AWAY"`, `"ARMED_STAY"` - `"since_armed_sec"`: `int`, seconds elapsed between arming and this event (ignored for `"panic"` and when `arm_state == "DISARMED"`)

- `exit_delay_sec`: `int`, default `45`, grace period after arming.

- `verify_window_sec`: `int`, default `120`, max gap (in `ts`) between two different-zone real trips for cross-zone verification to apply.

Output

A `list[str]` of classifications, **one per input event, in the same order as the input**. Each entry is one of `"SUPPRESSED"`, `"FALSE_ALARM"`, `"LOCAL_ONLY"`, `"DISPATCH"`.

Examples

```
events = [
    {"ts": 1000, "zone": "front-door", "sensor": "contact",
     "arm_state": "ARMED_AWAY", "since_armed_sec": 10},
]
solve(events)  ->  ["FALSE_ALARM"]

events2 = [
    {"ts": 1000, "zone": "z1", "sensor": "panic",
     "arm_state": "DISARMED", "since_armed_sec": 0},
]
solve(events2)  ->  ["DISPATCH"]

events3 = [
    {"ts": 1000, "zone": "z1", "sensor": "contact",
     "arm_state": "ARMED_AWAY", "since_armed_sec": 300},
    {"ts": 1050, "zone": "z2", "sensor": "glass_break",
     "arm_state": "ARMED_AWAY", "since_armed_sec": 305},
]
solve(events3)  ->  ["DISPATCH", "DISPATCH"]
```

Constraints

- Deterministic, offline, standard library only.

- Events may be given in any timestamp order in the list; the output order must match the **input** order (do not re-sort the output).

- Cross-zone verification: two real trips (armed, past exit delay) in **different** zones with `abs(ts_a - ts_b) <= verify_window_sec` corroborate each other. A real trip may be corroborated by more than one other zone; any single qualifying pair is enough to promote it to DISPATCH. Same-zone repeats never corroborate each other.

- `since_armed_sec >= exit_delay_sec` counts as past the exit delay (boundary is inclusive on the "past delay" side, i.e. exactly `exit_delay_sec` is a real trip, not a false alarm).

- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict


def solve(events: List[Dict], exit_delay_sec: int = 45, verify_window_sec: int = 120) -> List[str]:
    n = len(events)
    base = [None] * n
```

```
    for i, e in enumerate(events):
        if e["sensor"] == "panic":
            base[i] = "PANIC"
        elif e["arm_state"] == "DISARMED":
            base[i] = "SUPPRESSED"
        elif e["since_armed_sec"] < exit_delay_sec:
            base[i] = "FALSE_ALARM"
        else:
            base[i] = "REAL"

    real_indices = [i for i in range(n) if base[i] == "REAL"]

    result = [""] * n
    for i, e in enumerate(events):
        if base[i] == "PANIC":
            result[i] = "DISPATCH"
        elif base[i] == "SUPPRESSED":
            result[i] = "SUPPRESSED"
        elif base[i] == "FALSE_ALARM":
            result[i] = "FALSE_ALARM"
        else:
            corroborated = False
            for j in real_indices:
                if j == i:
                    continue
                if events[j]["zone"] == e["zone"]:
                    continue
                if abs(events[j]["ts"] - e["ts"]) <= verify_window_sec:
                    corroborated = True
                    break
            result[i] = "DISPATCH" if corroborated else "LOCAL_ONLY"

    return result
```

2.4.c · Motion detectors and sensor-based detection

3 pts · auto-graded on the site

Coding problem 2.4.c: Motion-sensor event classifier

Skill: Detect Attacks · **Maps to:** 2.4.B, 2.4.C · **Language:** Python 3 (pytest)

Problem

You are building the automated triage logic for a facility's motion-sensor monitoring system at Crestfield Analytics. Given a list of motion-sensor trigger events, classify each one as a **TRUE_DETECTION** (a real person likely caused it) or a **FALSE_POSITIVE** (a known nuisance cause, sunlight, HVAC airflow, a pet, an insect, vibration, or foliage, likely caused it), and return the IDs of the events that are true detections.

Single-technology sensors (`pir`, `microwave`, `ultrasonic`) report one cause tag. A **dual-technology** sensor reports two independent cause tags, one from its PIR element, one from its microwave element, and only counts as a true detection when **both** elements agree the cause is genuine (AND-logic). This mirrors the lesson: a dual-tech sensor exists specifically to suppress the false-positive causes that fool a single technology, so if *either* element's cause is a known false-positive cause, the whole event is a false positive.

Function contract

Implement `solve(events, false_positive_causes)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `events`: `list[dict]`, each dict has: - `"id"`: `str`, event identifier, e.g. `"EVT-001"` - `"sensor_type"`: `str`, `"pir"`, `"microwave"`, `"ultrasonic"`, or `"dual-tech"` - single-tech events also have `"cause"`: `str` - dual-tech events instead have `"cause_pir"`: `str` and `"cause_mw"`: `str`

- `false_positive_causes`: `set[str]`, known false-positive cause tags, e.g. `{"sunlight", "hvac_airflow", "pet", "insect", "vibration", "foliage"}`

Output

A **sorted list of unique event ID strings** that classify as TRUE_DETECTION. Empty list if none.

Examples

```
events = [
    {"id": "EVT-001", "sensor_type": "pir", "cause": "person"},
    {"id": "EVT-002", "sensor_type": "pir", "cause": "sunlight"},
]
fp = {"sunlight", "hvac_airflow", "pet", "insect", "vibration", "foliage"}
solve(events, fp)  ->  ["EVT-001"]

events2 = [
    {"id": "EVT-010", "sensor_type": "dual-tech", "cause_pir": "person", "cause_mw": "person"},
    {"id": "EVT-011", "sensor_type": "dual-tech", "cause_pir": "hvac_airflow", "cause_mw": "person"},
]
solve(events2, fp)  ->  ["EVT-010"]
```

Constraints

- Deterministic, offline, standard library only.

- Every event has the fields required for its `sensor_type` as described in `interface.md`.

- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import Dict, List, Set


def solve(events: List[Dict], false_positive_causes: Set[str]) -> List[str]:
    true_detections = []

    for e in events:
        sensor_type = e["sensor_type"]

        if sensor_type == "dual-tech":
            pir_ok = e["cause_pir"] not in false_positive_causes
            mw_ok = e["cause_mw"] not in false_positive_causes
            is_true = pir_ok and mw_ok
        else:
            is_true = e["cause"] not in false_positive_causes

        if is_true:
            true_detections.append(e["id"])

    return sorted(set(true_detections))
```